Research Statement

Yanbo Zhou

The long-term vision of my research is to build massive, cost-effective, and high-performance computing infrastructure to support future artificial general intelligence. My work addresses emerging challenges in memory and storage systems, where growing resource demands increasingly constrain system scalability and cost efficiency.

Emerging applications, such as large language models and AI agents, are placing unprecedented demands on computing infrastructure. Their rapidly growing memory and storage requirements are making infrastructure resources increasingly scarce and expensive, while constraining system scalability. In response, the semiconductor industry is developing new hardware technologies and expanding manufacturing capacity to meet these growing demands. These pressures also drive innovations in system software that can scale with evolving hardware to deliver the necessary capacity and throughput.

To address these challenges, my current work focuses on memory and storage systems in large-scale cloud infrastructure from three aspects: 1) advancing computer systems to fully exploit emerging hardware technologies; 2) rearchitecting computer systems by leveraging new hardware capabilities; and 3) innovating new hardware-software co-designed system architectures. Looking forward, my future work explores how AI agents will reshape the systems we build across cloud and edge environments and the way we build systems.

Advancing Computer Systems for Emerging Hardware Technologies

Emerging hardware provides significant improvements in capacity and cost-effectiveness, but these advantages do not automatically translate into real-world benefits because existing system software is often designed around legacy hardware assumptions. The first aspect of my research focuses on advancing computer systems to fully exploit the potential of emerging hardware technologies.

CSAL: a new storage tier for high-density SSDs. High-density SSDs provide greater storage capacity at lower cost than previous-generation SSDs, but they also pose new challenges to system software since higher-density flash offers lower endurance and requires coarser indexing units to reduce memory overhead. As a result, existing systems suffer from lower performance and reduced storage endurance, limiting the deployment in large-scale infrastructure. CSAL [1] introduces a new storage tier that places high-performance SSDs between applications and high-density SSDs to reshape user writes into flash-friendly access patterns and absorb frequently updated data, thereby achieving both high performance and high endurance. CSAL enabled the deployment of high-density SSDs in large-scale cloud infrastructure and has since been adopted by multiple production storage systems.

CSAL differs from traditional caching and tiering approaches in two key ways. First, it moves SSD firmware-level indexing into host software and integrates it with tiering metadata, enabling finer-grained indexing while controlling memory overhead through page tables and swapping. Second, it adopts log-structured data management across both fast and slow tiers, transforming small random writes from applications into large sequential writes on high-density SSDs and managing firmware-level flash garbage collection in software. Together, these designs address the performance and endurance limitations of high-density SSDs while preserving their capacity and cost advantages.

Sandman: a new CPU scheduler for large-capacity storage servers. Recent advances in storage servers have focused on delivering higher performance and greater capacity. However, these improvements can lead to significant energy consumption because high-performance storage systems require an increasing number of dedicated polling CPU cores for I/O processing, limiting their scalability. Sandman [2] is a CPU scheduler designed for modern storage systems that delivers both high performance and energy efficiency. Sandman establishes CPU-side energy efficiency as a new dimension of storage sustainability in large-scale cloud environments.

Sandman makes two unique contributions compared with existing CPU schedulers. First, to rapidly scale compute resources, Sandman leverages hardware cache coherence to wake sleeping CPU cores, avoiding both software and hardware wakeup overhead. Second, to detect I/O bursts at the microsecond timescales, Sandman monitors incoming I/O requests from network queues and builds a burst-detection model based on the number of incoming I/O requests, enabling finer-grained I/O burst detection while avoiding the inaccuracy introduced by CPU cycle-based detection.

Reachitecting Systems by Leveraging New Hardware Capabilities

Emerging hardware not only offers improvements in performance and cost efficiency but also introduces new capabilities that can fundamentally change how computer systems are organized. The second aspect of my research focuses on rearchitecting system abstractions to leverage these new hardware capabilities.

RamRyder: a software-defined elastic memory system for the cloud. Cloud platforms offer elastic compute, storage, and networking resources, but memory allocation remains inflexible: cloud providers typically allocate memory capacity in a fixed ratio to virtual CPUs and offer no mechanism for bandwidth allocation. This mismatch can lead to underutilization of both memory capacity and bandwidth as application demands diverge or change over time. RamRyder [3] is a new memory system that enables mostly independent allocation of memory bandwidth and capacity, with robust performance isolation, by leveraging the memory-channel capabilities of DDR and CXL memory. RamRyder establishes a new direction in software-defined, "open-channel" memory systems and was evaluated using large-scale cloud traces to demonstrate its performance and utilization benefits under realistic cloud workloads.

The key insight of RamRyder is to manage memory at channel granularity. Modern memory architectures provide an increasing number of DDR channels for higher bandwidth, while CXL memory devices also contain internal DDR channels that provide additional bandwidth. RamRyder therefore shifts memory-channel management from hardware to software, directly controlling channel allocation and page-to-channel mapping. To enable this design, RamRyder introduces a new channel abstraction in the operating system that supports page-to-channel allocation. Furthermore, RamRyder introduces new page-to-channel allocation policies across two tiers—local DIMMs and CXL devices—enabling mostly independent extension of memory capacity and bandwidth.

Aspen: an efficient sandbox forking mechanism across distributed nodes. Emerging agent applications require cloud infrastructure to rapidly scale sandbox instances across distributed nodes. Existing remote-forking approaches rely on distributed storage or RDMA networks, incurring substantial software and hardware overhead. Aspen [4] is a new sandbox forking mechanism that scales sandboxes across distributed nodes over CXL fabrics. It reduces both software and hardware overhead by leveraging the low access latency of CXL memory and enabling zero-copy

sandbox forking on cross-node shared memory. Aspen is designed and implemented on CXL 2.0 hardware and is the first sandbox forking mechanism demonstrated on real hardware.

The key idea behind Aspen is a distributed in-memory filesystem abstraction. Aspen separates sandbox virtual memory management from physical memory mapping in the shared memory pool. The Aspen Filesystem (AspenFS) manages physical memory mappings using a range-based radix index and uses reference-based file cloning for memory snapshot generation and restoration, enabling zero-copy throughout the entire sandbox forking process. To maintain consistency across distributed nodes connected through a CXL 2.0 shared memory pool, AspenFS introduces a low-latency export-acquire protocol for consistent file sharing that flushes CPU cache lines only for dirty pages and parallelizes CPU cache writeback and invalidation across multiple cores.

Innovating Software-Hardware Co-Designed System Architectures

Rapidly growing application demands and advancing hardware performance place increasing pressure on infrastructure system software due to constrained computing resources. The third aspect of my research focuses on innovating software-hardware co-designed system architectures by rethinking the boundary between software and hardware to address this challenge.

Ristretto: an ASIC/SoC co-designed stack for cloud storage. Storage hardware performance has improved rapidly, with IOPS doubling every three years. However, modern cloud storage stacks are increasingly constrained by the limited CPU resources available to sustain this performance. Ristretto [5, 6] introduces a cloud-native storage architecture that combines ASIC acceleration with ARM SoCs to achieve high performance while preserving flexibility. Compared with traditional CPU-based storage stacks, Ristretto delivers higher performance with substantially fewer CPU resources. Ristretto demonstrates the feasibility of such a hardware-software co-designed storage architecture by deployment in large-scale cloud infrastructure and received a Best Paper Award for its insights into rethinking the software-hardware boundary.

The key insight behind Ristretto is to distinguish which logics should be offloaded to hardware for efficiency and which should remain in software for flexibility. This design is motivated by several years of experience deploying storage systems in large-scale cloud infrastructure. Ristretto offloads compute-intensive but relatively stable functions, e.g., virtualization backends and device drivers, to ASICs, while leveraging SR-IOV to reduce virtualization overhead caused by context switches. At the same time, because cloud storage requires customizable features for cost efficiency and multi-tenant quality of service, Ristretto uses SoCs with low-power ARM cores to implement a programmable block-abstraction layer for cloud-specific functionality.

Future Directions: From Traditional Cloud to Agentic Infrastructure

AI agents are becoming increasingly important and creating a new frontier for computer systems research. They not only drive specialized system designs for agentic infrastructure, including cloud and edge computing, but also offer new tools for building and optimizing computer systems. My future research will explore both sides of this relationship: systems for agents and agents for systems.

Agent-native systems across cloud and edge. Today's infrastructure systems are largely designed for traditional cloud workloads, such as databases, big-data analytics, and serverless computing. AI agents introduce a fundamentally different class of workloads: they dynamically generate and execute code, invoke tools, maintain state, communicate with other agents, and exhibit highly variable resource demands. These characteristics raise new systems research

questions. How should cloud infrastructure elastically allocate resources while providing strong isolation across large populations of AI agents? How should agent execution and state move across heterogeneous cloud and edge resources?

The first direction of my future research is to explore agent-native systems across clouds, edge computing, and the interactions between them. The goal is to create an efficient computing continuum in which agentic applications can seamlessly use resources ranging from local edge devices to large-scale cloud datacenters. Rather than treating agents simply as another workload running on existing cloud infrastructure, I will explore systems that treat AI agents as first-class entities and exploit their execution structure to improve efficiency, elasticity, and isolation. At the edge, resources are more constrained and applications often interact directly with sensitive user data. I will explore how agent workloads can be partitioned across cloud and edge by identifying the advantages of different environments at different stages of agent execution and dynamically distributing computation and state between them.

Agent-powered system engineering and optimization. System software must continuously evolve to support emerging hardware and applications. However, building new systems or even adding new features to existing ones is difficult and error-prone, especially as systems grow increasingly complex. Traditional system development and optimization require substantial human effort and long development cycles. Today's AI agents have already shown strong capabilities in coding and research across many domains, creating new opportunities for systems development and optimization. However, system software is extremely complex, and even small implementation errors can have severe consequences at the datacenter scale. This creates exciting opportunities, but also significant challenges.

The second direction of my future research will explore agent-powered system engineering and optimization. The ultimate goal is to enable automated system development and innovation to reduce human effort in building complex systems. To achieve this goal, I plan to first raise the abstraction level of system implementation by decoupling core system components from specialized features, allowing AI agents to operate above this abstraction without directly modifying critical components that could compromise system stability. Next, I will explore system specifications as "prompts" for AI agents, enabling agents to better understand system requirements and address complex systems problems. Finally, I will investigate new verification and testing techniques to ensure the correctness and robustness of agent-generated system code.

Summary. Overall, my prior research in memory and storage systems for large-scale cloud infrastructure provides a strong foundation for pursuing these future directions. I aim to push the frontier of systems research while developing practical technologies that can have real-world impact. I look forward to contributing to a collaborative and diverse research environment, working with students and researchers, and helping shape the next generation of computing systems.

References

[1] **Yanbo Zhou**, Erci Xu, Li Zhang, Kapil Karkra, Mariusz Barczak, Wayne Gao, Wojciech Malikowski, Mateusz Kozlowski, Łukasz Łasek, Ruiming Lu, Feng Yang, Lilong Huang, Xiaolu Zhang, Keqiang Niu, Jiaji Zhu, and Jiesheng Wu. CSAL: The Next-Gen Local Disks For The Cloud. In *Proceedings of the 19th ACM European Conference on Computer Systems (EuroSys)*, 2024.

[2] **Yanbo Zhou**, Erci Xu, Anisa Su, Jim Harris, Adam Manzanares, and Steven Swanson. Sleeping With One Eye Open: Fast, Sustainable Storage With Sandman. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, 2025.

[3] **Yanbo Zhou**, Erci Xu, Dongjoo Seo, Adam Manzanares, and Steven Swanson. Break On Through To The Other Side: Pooling Memory Elastically With RamRyder. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026.

[4] **Yanbo Zhou**, Jinyong Moon, Anisa Su, Kyumin Park, Dongjoo Seo, Erci Xu, and Steven Swanson. From Fork To Fleet: Scaling Agent Sandboxes Over CXL Fabrics With Aspen. In Submission.

[5] Leping Yang, **Yanbo Zhou**, Gong Zeng, Li Zhang, Saisai Zhang, Ruilin Wu, Chaoyang Sun, Shiyi Luo, Wenrui Li, Keqiang Niu, Xiaolu Zhang, Junping Wu, Jiaji Zhu, Jiesheng Wu, Mariusz Barczak, Wayne Gao, Ruiming Lu, Erci Xu, and Guangtao Xue. Here, There And Everywhere: The Past, The Present And The Future Of Local Storage In Cloud. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST)*, 2026. **Best Paper Award**.

[6] Erci Xu, Leping Yang, **Yanbo Zhou**, Gong Zeng, Li Zhang, Saisai Zhang, Ruilin Wu, Chaoyang Sun, Shiyi Luo, Wenrui Li, Keqiang Niu, Xiaolu Zhang, Junping Wu, Jiaji Zhu, Jiesheng Wu, Mariusz Barczak, Wayne Gao, Ruiming Lu, and Guangtao Xue. Cloud Local Storage Goes Elastic: From Hardware Offloading To Hybrid Architecture. *ACM Transactions on Storage (TOS)*, 2026. **Invited Paper (extended version for FAST'26)**.